

Module 6: Spring Data JPA with Spring Boot, Hibernate

1. Spring Data JPA – Quick Example

Spring Data JPA is a module of the Spring Framework that simplifies database access by reducing the amount of boilerplate code required for data persistence. It builds on top of JPA (Java Persistence API) and usually uses Hibernate as the default JPA implementation.

With Spring Data JPA, developers only need to create an entity class and a repository interface. Common database operations such as Create, Read, Update, and Delete (CRUD) are automatically provided without writing SQL queries.

Example

Entity Class (Book.java)

```
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
@Entity
public class Book {
    @Id
    private int id;
    private String title;
    private String author;
    public Book() {
    }
    public Book(int id, String title, String author) {
        this.id = id;
        this.title = title;
        this.author = author;
    }
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getTitle() {
        return title;
    }
    public void setTitle(String title) {
        this.title = title;
    }
    public String getAuthor() {
        return author;
    }
    public void setAuthor(String author) {
        this.author = author;
    }
}
```

```
}  
}
```

Repository Interface (BookRepository.java)

```
import org.springframework.data.jpa.repository.JpaRepository;  
public interface BookRepository extends JpaRepository<Book, Integer> {  
}
```

Service Class

```
import org.springframework.beans.factory.annotation.Autowired;  
import org.springframework.stereotype.Service;  
@Service  
public class BookService {  
    @Autowired  
    private BookRepository repository;  
    public void saveBook(Book book) {  
        repository.save(book);  
    }  
    public void displayBooks() {  
        System.out.println(repository.findAll());  
    }  
}
```

Main Application

```
import org.springframework.beans.factory.annotation.Autowired;  
import org.springframework.boot.CommandLineRunner;  
import org.springframework.boot.SpringApplication;  
import org.springframework.boot.autoconfigure.SpringBootApplication;  
@SpringBootApplication  
public class LibraryApplication implements CommandLineRunner {  
    @Autowired  
    private BookService service;  
    public static void main(String[] args) {  
        SpringApplication.run(LibraryApplication.class, args);  
    }  
    @Override  
    public void run(String... args) {  
        Book book = new Book(101, "Spring Framework", "Rod Johnson");  
        service.saveBook(book);  
        service.displayBooks();  
    }  
}
```

Output

[Book(id=101, title=Spring Framework, author=Rod Johnson)]

Advantages of Spring Data JPA

- Reduces boilerplate code.
- Automatically provides CRUD operations.
- Supports custom query methods.

- Integrates seamlessly with Spring Boot.
- Improves productivity and maintainability.

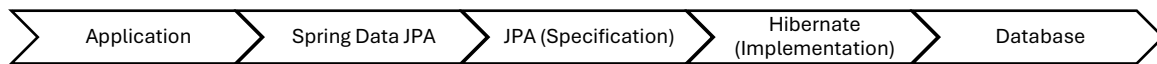
Conclusion

Spring Data JPA simplifies database interaction by automatically implementing repository methods for CRUD operations. It enables developers to focus on business logic instead of writing repetitive database code.

2. Difference between JPA, Hibernate and Spring Data JPA

Feature	JPA	Hibernate	Spring Data JPA
Definition	A Java specification (API) for Object Relational Mapping (ORM).	An ORM framework that implements JPA.	A Spring module that simplifies JPA-based data access.
Type	Specification	Framework	Spring Framework Module
Developed By	Oracle (Java Community Process)	Hibernate Team (Red Hat)	Spring Team
Purpose	Defines standards for ORM.	Implements ORM functionality and JPA specification.	Simplifies database access using JPA repositories.
SQL Handling	Does not generate SQL by itself.	Generates SQL automatically.	Uses JPA/Hibernate to generate SQL.
CRUD Operations	Must be written using EntityManager.	Uses Session or JPA EntityManager.	Provides built-in CRUD methods through JpaRepository.
Boilerplate Code	More	Moderate	Very Less
Query Support	JPQL	HQL, JPQL, Native SQL	Derived Query Methods, JPQL, Native SQL
Dependency	Requires an implementation such as Hibernate.	Can work independently or as a JPA provider.	Depends on both Spring Framework and a JPA provider like Hibernate.
Ease of Use	Moderate	Easy	Very Easy

Relationship between JPA, Hibernate, and Spring Data JPA



Conclusion

- **JPA** defines the standard rules for Object Relational Mapping (ORM) but does not provide an implementation.
- **Hibernate** is a popular ORM framework that implements the JPA specification and handles communication with the database.
- **Spring Data JPA** is built on top of JPA and further simplifies data access by automatically providing repository implementations and CRUD operations, resulting in cleaner and more maintainable code.